

Test Plan: Vehicle Rental System

Course: Software Testing and Validation

1. Scope

The scope of testing encompasses the core functionality of the new Vehicle Rental System web application. This includes: * Core business logic: Age restrictions, rental duration validation, discount application, and total price calculation. * State management: Validating correct booking state transitions (Requested -> Confirmed -> Active -> Returned). * User Interface: Testing the end-to-end user journey for booking a vehicle via the web interface. * *Out of scope:* Load testing, security penetration testing, and payment gateway integration.

2. Approach

Testing follows the traditional Test Pyramid strategy: * **Unit Testing (pytest)**: Tests individual methods in isolation (e.g., RentalCalculator). We utilize mocks and test doubles where necessary. * **Integration Testing (pytest-django)**: Tests the interaction between the Django Models, database, and business logic. * **System Testing (Playwright)**: End-to-end testing of the complete user journey through the browser, utilizing the Page Object Model (POM) pattern. * **Acceptance/UAT**: Confirm that a customer can book an available vehicle and that invalid age and duration inputs produce understandable errors. * **Continuous Integration**: Automated execution of all test levels via GitHub Actions and Jenkins pipelines on every code change.

3. Entry and Exit Criteria

Entry Criteria: * Unit code compiles without syntax errors. * Test environment is fully provisioned (Docker containers active). * Test design documents and cases are approved.

Exit Criteria: * 100% of planned test cases executed. * No high-severity or critical defects remain open. * Code coverage achieves a minimum of 80% branch coverage on core logic. * Test Summary Report is complete and signed off.

4. Risk-Based Prioritization

Testing effort will be prioritized based on the following risk profile: 1. **High Risk (Test Hardest)**: RentalCalculator (financial impact if incorrect) and BookingManager (data corruption if state is invalid). 2. **Medium Risk**: Form validation on the UI (usability impact). 3. **Low Risk**: Static template rendering (visual impact only).

5. Schedule

- **Week 1:** Requirements analysis, architecture setup, Test Plan creation.
- **Week 2:** Unit and Integration test implementation, Jenkins/GitHub Actions CI setup.
- **Week 3:** Playwright E2E tests, Defect tracking, Metrics calculation, and Final Reporting.

6. Roles and Responsibilities

- **Eyob Mulugeta:** Test lead and developer for business logic, unit/integration tests, and CI/CD configuration.
- **Afomia Birhanu:** Quality analyst for EP/BVA, decision tables, state transitions, Playwright Page Objects, UAT, defects, and reporting.